Planstudio v2: a CAD-style kernel in Rust, MCP-first, with a SketchUp-style desktop editor and a mobile viewer

Context

v1 (this repo) stores a wall as a free centre line `a→b` plus a thickness. Joints are two wall ends within 1 mm of each other, openings sit at an `offset` along `wall.a`, room areas come from a raster flood fill, and a `Dimension` is two loose points that drive nothing. That fits tracing a scan. It fits badly the work of drawing from laser measurements, which are clear sizes between wall faces.

Drawing lgh 154 showed three concrete failures:

- Face measurements have to be converted to centre lines by hand.
- Conflicting measurements (4.49 m against 2.36 + 0.09 + 2.09 = 4.54 m) are absorbed silently, as a south-wall step and 10 mm fudges.
- A guessed outer thickness can shift a room.

Calibrating the Gnejsen 152–154 basplan against new laser readings showed the same pattern at a larger scale. Chained measurements exposed 205 mm and 228 mm errors, an implausible 294 mm WC wall, and walls that should align but didn't. Each one took several manual rounds to find and fix.

Exploring v1 found about 600 call sites of the centre-line model in about 45 files, and no schema version on stored plans. A staged in-place refactor would spend most of its effort keeping the old model alive, so v2 is a **separate project**.

Decisions made with the user:

- **New repo, monorepo.** v1 keeps running until v2 reaches parity.
- **Language:** Rust throughout. The kernel compiles to WASM for the browser and to native code for the server and desktop.
- **Editor:** a SketchUp-style 2D desktop editor written in Rust (egui and wgpu), running natively and in the browser. It replaces the earlier TypeScript and React web editor; see "Research" for why.
- **Licence:** GPL-3.0, like v1.
- **Scan and trace tools:** stay in v1 as the import path first, and are ported last.

Architecture

Principle: one pure kernel, one command API, thin clients. Meaning (walls, rooms, measurements) is the source of truth, and geometry is computed from it.

1. **Kernel (`core`, no I/O).** Exact integer-mm geometry and the model:
    - **Nodes.** Walls reference nodes, so a joint is a shared node and not a 1 mm coincidence.
    - **Walls.** A wall has `from`, `to`, `thickness` and `ref: left|center|right`. `ref` says which face the node line is, so a wall measured from the inside keeps that face and a wrong thickness only grows outward.

    - **Openings.** Anchored to a wall end or centre with a distance and a width.
    - **Rooms.** Derived closed regions from the planar graph of wall faces, with exact polygon area and clear sizes.
    - **Dimensions.** Driving or reference. A driving dimension is a constraint. Phase 1 solves straight walls with a deterministic axis-by-axis graph solver that explains each conflict as two chains of numbers (see the agent specs addendum §6). SolidCraft's sketch solver (see "Research") sits behind the same interface for angled geometry, if the Phase 0 spike confirms it fits. Messages are in domain terms, for example "4.49 measured, 4.54 implied, 50 mm apart".
    - **Missing measurements.** The solver's degree-of-freedom analysis also finds what nothing pins down. The kernel turns that into a list of measurements to take next, for example "Garderob depth not measured; laser from the north wall to the partition". This follows Q12 (see "Research").
    - **Phases.** The per-element `phase` tag (`existing|demolish|new`) is carried over unchanged.
2. **Command API (the only way to change a plan).** A tagged `Command` enum, applied all-or-nothing, with a `dry_run` that returns a diff and diagnostics.
    - Commands are semantic: `SetClearSize`, `PlaceWall`, `MoveNode`, `SetDimension`, `AddOpening`.
    - Schemas come from the Rust types (`schemars`). JSON Schema generates the MCP tool schemas and the REST/OpenAPI definitions.
3. **Storage as a command log plus snapshots.** This gives undo, revision history, compare and provenance (agent or human) without extra machinery.
4. **Render-neutral scene.** The kernel emits primitives with semantic ids. Backends produce SVG (in Rust, so output is identical everywhere), PDF (via an SVG-to-PDF crate), DXF and SketchUp `.skp`.
5. **Three clients on the same API:**
    - **MCP server.** MCP-first: every result carries the diagnostics and, on request, a picture.
    - **Desktop editor.** Rust (egui and wgpu), native on Windows, macOS and Linux and in the browser through WASM. It links the kernel directly and sends the same commands. Dragging is a command preview.
    - **Mobile viewer.** A read-only PWA that loads the scene or SVG, with pan, zoom and share links. No edit code ships to it.

LLM-first interface design

Models are good at names, topology, small integers, editing structured text and fixing their input from a precise error. They are bad at absolute coordinates, long arithmetic, left/right conventions, angles and reading measurements off images. The interface is built around that.

1. **Intent layer is the source of truth.** The document stores what was measured: rooms with clear sizes, partitions with thickness, openings addressed by room and side. The kernel resolves coordinates, so a model never computes one. Resolved geometry is a build artefact. Every dimension carries `source: measured | printed | estimated | assumed` (and optional tolerance), so reports can say what was measured and what was estimated.
2. **A compact text DSL that round-trips.** `get` returns it and `apply` accepts patches to it. Example: `Garderob 1860×2360; Sovrum below Garderob, wall 90; Sovrum.south window 1350 from west 380`. Units are mm, with `1.86 m` parsed as well.

3. **Names and compass directions, not a/b and left/right.** `hinge south, opens into Vardagsrum`. Ids are semantic (`Garderob.east`). Commands upsert by name, so retries are safe.
4. **About 7 tools:** `apply` (all-or-nothing, with `dry_run`), `get` (views: summary, dsl, json, svg), `check`, `render` (labelled, zoomable, diagnostic views; see "Giving the model vision"), `find`, `import` (drawings, photos of laser displays, sketches), `history`. Verbs live in the DSL, so a cheap model has fewer choices. A basic profile exposes the minimum.
5. **Structured verification first, vision on demand.** `apply` returns the diff and diagnostics with numbers, plus suggested next steps. Labelled, zoomable renders are available when judgement is needed (see "Giving the model vision").
6. **Actionable errors.** "Unknown room 'Sovrun', did you mean 'Sovrum'?" and "over-constrained: 4.49 measured vs 4.54 implied; choose which to trust". Conflicts are where the model should ask the user.
7. **Axis-aligned first-class.** Angled walls only through an explicit angle.
8. **MCP details:** output schemas, read-only and idempotent hints, the plan as an MCP resource, and prompts for the working order. SolidCraft's `solidcraft-cli mcp`, which can drive the live app, is a reference design.
9. **Cost control.** Deltas after edits, not full state. Pagination and field selection on `get`. Small images only on request.
10. **Eval gate.** A fixed task set (for example draw Igh 154 from a measurement list) runs on Opus, Sonnet and Haiku, with pass rate and tokens per task. The target is that the cheapest tier passes. Benchmarks are varied and not cherry-picked, as HN commenters demanded of text-to-CAD tools.
11. **Next-step guidance.** `check` returns what to do next (missing measurements, unresolved conflicts, open review findings), like v1's `plan_next`. The wedge-ramp Ask HN found that splitting the work into ordered steps is the hard part for a model.
12. **Lessons from text-to-CAD threads** (see "Research"): give the model a verification loop and queryable summaries instead of full files, make frames and orientation explicit, run automated checks, and keep a context budget. Visual feedback is covered in "Giving the model vision".

Giving the model vision (researched 2026-11-09)

Principle: vision for judgement, numbers from the kernel. Benchmarks are consistent:

- On AECV-Bench, models read text on drawings well (up to 0.95) but count door and window symbols badly (0.40–0.55) (arXiv 2601.04819).
- A 2026 study found vision models' room-area estimates far worse than deterministic image analysis (Periodica Polytechnica).
- Anthropic's own docs say pixel positions Claude reports are approximate and should be checked (Vision, Coordinates).

So the model never measures from pixels. It looks at pictures to decide what is wrong, where to look next, or which element a photo or note refers to, and every number comes from the kernel or a laser.

Costs and limits:

- An image costs ⌈w/28⌉ × ⌈h/28⌉ tokens. Current models (4.7 and later) take up to 2,576 px on the long edge and 4,784 tokens per image.

- A full A3 sheet at maximum resolution is about 4.7k tokens, roughly $0.0005 on Haiku 5.5 and $0.02 on Opus 5.5.
- Small symbols are lost when a large image is scaled down, so detail needs crops, not bigger images.

What the MCP server provides:

1. **Pictures only on request.** `render` takes a view, an optional region (by element id or a rectangle in mm) and a token budget. Edits return text diagnostics. A picture is suggested only when the diagnostics say something is ambiguous.
2. **Labels on every render (set-of-marks).** Walls, nodes, rooms, openings and dimensions are drawn with their semantic ids or names. The model then refers to "wall Garderob.east" instead of a pixel position, and its answer becomes a command directly. Labelled overlays clearly improve how accurately vision models refer to parts of an image (Set-of-Mark prompting, arXiv 2310.11441, code).
3. **Exact pixel-to-mm mapping.** Renders are pre-sized to fit the model's limits, using Anthropic's resize rule and `transformations: {oversized_image: "error"}` so nothing is resized silently. Each render carries mm-per-pixel and the origin in its metadata, and draws a mm grid and scale bar, as v1's `plan_view` already does.
4. **Zoom like computer use's zoom action.** The model asks for a high-resolution crop around an id or rectangle, for example a door swing or a wall joint. Crops are the main tool, not full sheets.
5. **Diagnostic views that make the kernel's findings visible:**
   - **Conflict view:** walls coloured by how far they are from the measurements that place them.
   - **Missing-measurement view:** what isn't pinned down is highlighted, with suggested laser shots drawn as arrows.
   - **Before/after diff:** what an edit changed, in red and green.
   - **Overlay on the source:** the plan over a scan, photo or SketchUp screenshot, to check alignment.
6. **Reading user images into structured data.** Photos of a laser display, sketches and SketchUp screenshots are read with structured outputs into readings. A labelled render of the current plan comes alongside so the model can propose which wall each reading spans. Ambiguity goes back to the user; this session hit exactly that, with display order and wall faces.
7. **Human in the loop through MCP Apps.** When the model can't tell which wall a reading belongs to, it shows an interactive plan in the chat and the user taps the wall, instead of the model guessing (MCP Apps, stable since 2026-01-26). AECV-Bench's authors also recommend tool-augmented, human-in-the-loop workflows.
8. **Scan to vector, later.** Specialised models convert raster plans to vectors far better than general vision models; FloorplanVLM reports 92.5% exterior-wall overlap (arXiv 2602.06507). Evaluate one as the replacement for v1's scanner, with calibration kept deterministic.
9. **Eval.** The eval gate includes vision tasks (a laser photo plus notes, and spotting a misaligned wall in a labelled render), scored on correctness and image tokens used.

Models and cost (estimates, prices cached 2026-10-06, per 1M tokens)

Prices: Fable 5.1 $10 in / $50 out; Opus 5.5 $4 / $20; Sonnet 5.5 $2 / $10; Haiku 5.5 $0.10 / $0.50 (prompts up to 100K). Cache reads are about $0.20 on Opus 5.5 and Sonnet 5.5.

Building it:

- Opus 5.5 for the kernel, solver integration, command schema, DSL and architecture reviews.
- Sonnet 5.5 for the server and MCP adapter, editor, viewer, tests, importer and CI.
- Haiku 5.5 for fixtures, docs and mechanical edits.
- Fable 5.1 only if a design problem stalls (2.5× Opus 5.5).
- Per-phase sessions and costs are in "Effort and cost by phase".

Running it (LLM using the MCP tools), per drawing like lgh 154, assuming 150K in / 5K out uncached: Haiku 5.5 about $0.02, Sonnet 5.5 about $0.35, Opus 5.5 about $0.70. Prompt caching on the stable prefix cuts the input side by up to about 90%.

Runtime tiers: Haiku 5.5 is the floor the eval gate must pass, Sonnet 5.5 is the default, Opus 5.5 is for ambiguous cases (measurement conflicts, reading a photo).

API constraints that shape the tool design:

- Forced `tool_choice` (`any`/`tool`) returns a 400 on Opus 5.5 and Sonnet 5.5. Tools must work with `auto` plus `strict: true` schemas and clear descriptions.
- Thinking can't be disabled on Opus 5.5. Effort defaults to `medium` on Opus 5.5 and Haiku 5.5. The eval sweeps effort per tier.
- Keep Haiku prompts under 100K tokens, since beyond that its price is five times higher.

Research: existing open-source tools (checked 2026-11-09)

Prompted by the Hacker News thread "KetchUp – Open-source SketchUp alternative built with Rust and WebGPU". BenchUp and SolidCraft were cloned and their code read; the others were checked through their pages and listings.

BenchUp, formerly KetchUp: borrow from it, don't fork

github.com/BenchUp3D/editor. MIT, Rust, about 103k lines in two crates (`benchup-core` about 19k, `benchup-app` about 83k), committed daily.

- **What it has:**
  - A working SketchUp-style tool set: Select, Line, Rectangle, Push/Pull, Offset, Move, Rotate, Paint, Tape Measure, Protractor, Dimensions, Erase.
  - Snapping with red/green axis locks, endpoint, midpoint and on-edge.
  - Groups, components, tags, an outliner, undo history, autosave, and a top orthographic view.
  - A typed-value input that understands units and formulas (`60cm+89mm`).
  - Native desktop and browser builds from one Rust codebase (egui interface, wgpu rendering, no JavaScript framework).
  - SketchUp `.skp` import through OpenSKP.
- **Why not fork:**
  - Its model is free-form 3D geometry (vertices, edges, faces with holes), exactly like SketchUp, and the tools edit that geometry directly. Line adds edges, Push/Pull extrudes faces into prisms, and a Dimension is a label pinned to two vertices that changes nothing.
  - In v2 the geometry is computed from walls, rooms and measurements. The same tools must create and move walls, keep openings attached, re-derive rooms and solve driving

dimensions. The part of every tool that decides what a click or typed value does to the document has to be rewritten.
    - Tagging raw faces as "walls", as SketchUp architecture plugins do, breaks what this project needs. Nothing stops a corner drag from making a wall non-parallel or the wrong thickness. Laser measurements can't drive geometry. Room areas, permit checks and före/efter sheets need meaning the geometry doesn't hold.
    - About 57 files are woodworking-specific (joinery, grain, stock, cutting plans, fasteners, time estimates). Accounts and sharing depend on the author's PHP server.
    - The code is tightly coupled (`app.rs` 6.5k lines, `renderer.rs` 7.7k, `sidebar.rs` 5k), and it changes daily, so a fork would drift fast.
- **What to lift, keeping MIT attribution:**
    - The snapping engine (`benchup-core/src/inference.rs`, about 1.5k lines). It is 3D but already supports snapping on a plane and works through an abstract `FaceQuery` interface over the geometry.
    - The typed-value parser (`benchup-app/src/precision.rs`), dimension rendering, undo history and shortcuts.
    - The egui and wgpu application setup as a template.
- **Benchmark:** BenchUp is the yardstick for how SketchUp-like the editor feels. HN commenters found its Push/Pull, default snapping and rectangle naming felt unlike SketchUp. That is why the acceptance test below is "a SketchUp user without docs".

SolidCraft: adopt its sketch constraint solver

github.com/tangency-labs/solidcraft. MIT or Apache-2.0, Rust, a parametric Fusion-360-style CAD with a built-in MCP server, about 2.5M lines.

- Its `solidcraft-sketch` crate (`crates/sketch`, about 33k lines, `#![forbid(unsafe_code)]`) is a 2D constraint solver:
    - points and curves, geometric constraints and **driving dimensions**;
    - a Levenberg–Marquardt solver per connected component;
    - degree-of-freedom analysis that detects over- and under-constrained sketches;
    - a constructive seed that keeps a sketch from flipping into a mirrored solution when a dimension changes.
- Its dependencies are small: `solidcraft-geom` and `solidcore-record` (NVIDIA, Apache-2.0). It already runs in SolidCraft's browser app.
- **Why adopt it:** it replaces both the in-house axis-aligned solver and the GPL alternatives (SolveSpace's solver, FreeCAD's `planegcs`) from the earlier plan. It is also GPL-3.0-compatible with no relinking burden in the WASM bundle.
- **Risk:** it lives inside a very large, fast-moving repo. Vendor a pinned copy of the three crates behind a thin adapter in our kernel, so it can be swapped out.

OpenSKP: SketchUp import and export

github.com/iamahsanmehmood/openskp. MIT, reverse-engineered, not affiliated with Trimble. C++17, TypeScript, Python, .NET and Dart.

- Reads SketchUp files from 2013 onward (both the classic and the 2021+ container). Per its README it also **writes** `.skp`, in the 2013–2020 container.

- **Why it matters:** SketchUp for Web on the free plan imports only `.skp`, `.jpg` and `.png`; DXF and DWG import need a paid plan (SketchUp help: file management). A native `.skp` export is therefore what lets the user open plans in their SketchUp. OpenSKP avoids depending on Trimble's hosted SketchUp service.
- **Verify in the Phase 0 spike:** a file OpenSKP writes opens in SketchUp for Web, with walls, tags and groups intact.

Related Hacker News discussions (searched 2026-11-09)

- **Show HN: OpenPlan – 2D CAD for floor plans, in the browser** (openplan.am): the closest competitor.
  - **Has:** walls with typed lengths and real thickness that join cleanly at corners, doors and windows that cut walls, rooms that appear when walls close with live areas, snapping, a check for broken wall joints, DXF import and export, a furniture library. Free with no account.
  - **Lacks, per its homepage:** measurement-driven dimensions, permit checks, AI, and any stated licence or repo.
  - **Use:** the UX reference for drawing walls and rooms.
- **Show HN: Q12 – A constraint-based 2D drawing tool** (q12.app): inequality, area and expression constraints. One of its examples is checking land areas against survey dimensions and identifying missing measurements. That's the source of the kernel's missing-measurement report.
- **SolveSpace – Parametric 2D/3D CAD**, also praised in the Nasscad thread: repeatedly named the most stable, fastest sketcher and solver among open-source tools. It's GPL C++, so it's the benchmark to compare SolidCraft's solver against in the Phase 0 spike, not a dependency.
- **Show HN: SolidCraft**: commenters call it early-stage and worry about long-term maintenance of AI-built CAD. That supports vendoring a pinned copy behind an adapter. The repo moved from `bherbruck/solidcraft` to `tangency-labs/solidcraft`.
- **Text-to-CAD (earthtojake)** and **Ask HN: Why can AI generate Super Mario but not a wedge ramp?**: what works for LLM-driven CAD:
  - render-and-fix loops;
  - topology summaries instead of full files;
  - explicit frames and orientation;
  - automated geometric checks;
  - honest benchmarks;
  - breaking the work into ordered, specified steps run through MCP.
- **Smaller references:** Arxel (constraint-driven building masses), Cota (browser CAD with automatic dimensions).

Checked, not adopted

- **Hew** (listing): a SketchUp-like modeller where every shape is a solid. AGPL-3.0, early-stage with a small community, imports `.skp` only from 2017. Nothing it offers isn't already covered by BenchUp and OpenSKP.
- **"ingetrazo"** (a SketchUp clone with an MCP server, mentioned on HN): not found. The closest matches are MCP bridges that drive a desktop SketchUp you own, such as sketchup-mcp2.
- **react-planner** (cvdlab, MIT): its model is vertices, lines between shared vertices, holes with an `offset`, and auto-detected areas, with wall thickness only as a free-form property. There is no

face reference and no dimensions or constraints, and its state lives in Redux and Immutable.js, which conflicts with a kernel that owns the truth. Upstream looks stale; a TypeScript fork (`archef2000/react-planner`) shows October 2025 commits. It confirms explicit shared vertices as the right topology.
- **Trimble's SketchUp MCP connector:** a hosted service that runs Python against a live SketchUp model and returns a `.skp` download link. It works today and is the fallback if OpenSKP's writer falls short, but it is a paid, external dependency.

Licence

- v2 is **GPL-3.0**, like v1 (decided with the user).
- The adopted pieces are all compatible: BenchUp (MIT), SolidCraft (MIT or Apache-2.0), solidcore-record (Apache-2.0), OpenSKP (MIT).
- Keep each upstream's licence file and attribution alongside the vendored code.
- Browser delivery of the WASM editor counts as distribution, so GPL source must be offered alongside it. Record this in the README and the release process.

Reuse, don't build

Phase 0 includes a spike that verifies each item (current licence, WASM compatibility, maintenance) before adoption.

- **Constraint solver:** SolidCraft's `solidcraft-sketch`, vendored and pinned behind an adapter.
- **Editor interaction:** from BenchUp, the snapping engine, typed-value parser, dimension rendering, undo history, shortcuts and egui/wgpu setup.
- **SketchUp files:** OpenSKP for `.skp` export, and import as a reference underlay.
- **Auth, accounts, sharing, MCP OAuth:** an auth provider with OAuth 2.1 support (for example WorkOS AuthKit, Clerk or Auth.js). Do not port v1's home-built accounts, invitations and OAuth.
- **Geometry:** `geo`, `i_overlay`, `cavalier_contours` for union, area, offsets and containment. Rooms are the holes of the union of wall polygons, so no custom half-edge code.
- **Types and contracts:** `serde`, `schemars`, `utoipa`. **WASM:** `wasm-bindgen`, `trunk` (as BenchUp uses).
- **MCP and HTTP:** the official Rust MCP SDK (`rmcp`) and `axum`.
- **DSL parser:** `winnow` or `chumsky`.
- **Export:** `svg2pdf` and `resvg` for PDF and PNG, the `dxf` crate for DXF, OpenSKP for `.skp`, IFC (IfcOpenShell) later as an export only.
- **Storage:** SQLite or libSQL with a command-log table (v1 already uses Turso). Blobs on Vercel Blob, S3 or R2. Managed hosting.
- **Mobile viewer:** the kernel's SVG scene with a pan/zoom library. No custom rendering engine.
- **Collaboration (later):** Yjs, Automerge or Loro, or server-authoritative commands.
- **Scan pipeline:** OpenCV, and check for pretrained floor-plan parsing models before porting v1's custom scanner.
- **Tests and evals:** `proptest`, `insta`, Playwright for the viewer, egui's test harness for the editor, and an existing eval harness instead of a custom one.
- **Stays custom:** the intent layer and command API, the wall/room/opening model, how each editor tool maps to kernel commands, over-constraint messages, the scene model and the Swedish

permit checks.

Desktop editor: SketchUp-style 2D

The user knows SketchUp and wants the same experience, tools and shortcuts on desktop, in 2D. The editor copies SketchUp's interaction model, much of it lifted from BenchUp. Underneath, every tool produces kernel commands on walls, rooms and openings, instead of editing free lines and faces. Desktop first, mouse and keyboard; touch is secondary, and mobile stays view-only.

SketchUp tool (key)	v2 tool, same key by default
Select (Space)	Select walls, rooms, openings, fixtures. Double-click a room to select its walls.
Entity Info	Inspector panel: thickness, reference face, phase, flags, measured values and their source
Line (L)	Wall: click-click chain. Type a length in the Measurements box, or `length;thickness`. Tab or the arrow keys pick the reference face (left/centre/right).
Rectangle (R)	Room by clear size: type `1860,2360` and the walls are placed around it at the chosen thickness
Push/Pull (P)	Drag a wall face to resize a room; type the distance or the new clear size. Moves the wall, keeps its openings, re-derives the rooms.
Offset (F)	New parallel wall at a typed distance from a face (for example a partition 1060 from the north face)
Move (M)	Move walls, openings, fixtures. Ctrl to copy, arrow keys to lock an axis. Joined walls follow through shared nodes.
Rotate (Q)	Fixtures and furniture; walls only through an explicit angle
Tape Measure (T)	Measure and place guides. A "record" toggle stores the reading as a dimension with `source: measured`. Also used to scale an imported photo or scan, like SketchUp's resize trick.
Dimension (D)	Place a dimension. Double-click the value to edit it: a driving dimension runs the solver and moves the geometry, a reference one only displays. Conflicts are shown with the numbers.
Eraser (E)	Delete. Deleting a wall merges its two rooms and removes its openings.
Paint Bucket (B)	Paint the phase (existing, demolish, new) onto walls and openings
Tags	Layers: före/efter, furniture, dimensions, underlay
Scenes	Saved views: Före, Efter, permit sheet, working view
Components / 3D Warehouse	Fixture and furniture library (kitchen, WC, shower, beds, wardrobes)
Groups / Outliner	Apartments and rooms as groups; an outliner tree of apartments, rooms, walls
Measurements box	Always active, bottom right: type without clicking first. Accepts mm, `1.86 m` and formulas (from BenchUp's parser).

Inference and navigation:

- SketchUp-style coloured hints, adapted from BenchUp's engine to wall faces and corners: endpoint, midpoint, on face, red/green axis locks, parallel and perpendicular, and dotted inference lines from points you hover over. Shift locks the current inference.
- Scroll to zoom at the cursor, middle-drag to pan, Shift+Z for zoom extents.
- Esc cancels, and Ctrl+Z / Ctrl+Y undo and redo.
- Shortcuts can be changed; SketchUp's are the defaults.

Acceptance: a SketchUp user redraws Igh 154 from the laser measurements in under 15 minutes without reading docs, and any conflicting measurements are flagged on the way.

Repo layout (new repo)

- `crates/core`: model, faces, commands, diagnostics, and the solver adapter
- `crates/render`: scene, SVG, PDF, DXF
- `crates/skp`: `.skp` export (and underlay import) via OpenSKP
- `crates/checks`: ports of v1's `plan_review`, `plan_check` and structure rules
- `crates/import-v1`: reads v1 revisions (centre-line JSON) and builds v2 plans
- `crates/server`: HTTP and MCP adapter and storage
- `crates/editor`: the SketchUp-style editor (egui and wgpu), built native and for the browser with `trunk`
- `crates/cli`: apply commands to a plan, render, run acceptance cases
- `vendor/`: pinned copies of `solidcraft-sketch`, `solidcraft-geom`, `solidcore-record` and the lifted BenchUp modules, each with its licence and source commit
- `viewer/`: mobile PWA

Phases

Phase 0: Spec, acceptance cases and reuse spike

- Write the model spec and invariants.
- Turn real cases into fixtures: Igh 154 from the laser measurements, the calibrated Gnejsen 152–154 basplan (rev 40), v1's `flat`, `golden` and `cellar` fixtures, and the 153/154 plan (rev 50).
- Spike, one or two days each:
  - Solve Igh 154's walls as a SolidCraft sketch, including the 4.49 against 4.54 conflict. The main question is how walls with a thickness and a reference face map onto the solver's points and lines, so this spike decides the shape of Phase 1.
  - Snap on a 2D plan with BenchUp's inference engine.
  - Write a `.skp` with OpenSKP and open it in SketchUp for Web. OpenSKP is C++ and TypeScript, so also test building it through `cc` as BenchUp does.
  - Return a labelled, pre-sized render from v1's `plan_view` to Claude and check it names walls correctly. This can be prototyped in v1 now.
- Precondition: the user creates an empty repo. This session's GitHub scope is only `pasilun/laughing-tribble`, so I can't create one, but I can add it with `add_repo` once it exists.

Phase 1: Kernel and CLI

- Model, face geometry with exact mitre and T-joins, command application, the solver adapter and conflict diagnostics.
- Property tests (`proptest`): joints stay closed, a wall's faces stay parallel at its thickness, and commands are all-or-nothing.
- Intent layer and the text DSL (parser, printer, round trip) with `source` and tolerance on dimensions. Turning relational statements ("Sovrum below Garderob, wall 90") into solver constraints is the largest single piece of work in the project.
- Missing-measurement report from the solver's degree-of-freedom analysis.

- **Acceptance:** lgh 154 built from clear sizes and 90 mm partitions only, written as a DSL document of a few lines. No south-wall step, no fudges, the 4.49 against 4.54 conflict is reported with its numbers, and changing an outer thickness moves no room.

Phase 2: Render and export

- SVG, PDF, DXF and `.skp` from the scene. `.skp` goes through OpenSKP: a C++ build for native, its TypeScript build in a worker for the browser.
- Labelled renders (set-of-marks), zoomed crops by element id or mm rectangle, and pre-sized PNGs with mm-per-pixel metadata (see "Giving the model vision").
- Compare against v1's output on the fixtures. Differences should be only exact mitres.

Phase 3: Checks, rooms and diagnostic views

- Port the review, permit and structure checks onto polygons.
- Exact areas and clear sizes replace the raster flood fill.
- Diagnostic views: conflict colouring, missing-measurement highlights with suggested laser shots, before/after diff, overlay on a source image.

Phase 4: Server and MCP

- The 7-tool surface described above, generated from the schema, with `dry_run`, structured diagnostics, on-demand renders and command-log storage.
- `import` of photos and sketches into structured readings, with a labelled render so the model proposes which wall each reading spans.
- MCP App for "tap the wall you measured" when a reading is ambiguous.
- Run the cross-tier eval (Opus, Sonnet, Haiku), including the vision tasks, here before the editor, because the tool design is the main risk for cheap models.

Phase 4b: Accounts, sharing and MCP OAuth

- An auth provider with OAuth 2.1 (see "Reuse, don't build"). Per-user plans, share links for the viewer, and OAuth for remote MCP clients. v1's accounts are not ported.

Phase 5: Desktop editor

- egui and wgpu app, native and browser, starting from BenchUp's setup. Lift the snapping engine, typed-value parser, dimension rendering, undo history and shortcuts. Draw the plan with egui's 2D vector painter from the kernel's scene; BenchUp's 3D renderer is not needed.
- Packaging and distribution: installers for Windows, macOS and Linux (BenchUp uses Velopack), code signing and macOS notarisation, and the browser build.
- Each tool maps to kernel commands as in the table above. Dragging previews a command, and typed dimensions drive geometry through the solver.
- Build Select, Wall (L), Room (R), Push/Pull, Tape Measure and Dimension first, then the rest.
- Acceptance: the SketchUp-user test above.

Phase 6: Mobile viewer

- Read-only PWA with pan, zoom, share links and offline view.

Phase 7: Import and cut-over

- `import-v1` converts old plans, and the v1 scan tools feed v2 through it.
- Cut-over criteria: lgh 154, the basplan and the golden, flat and cellar fixtures round-trip and render within tolerance, and the MCP tool set covers v1's editing flow.

Later: scan pipeline

- Port or replace the scan and trace tools. A Python/OpenCV service that calls the kernel's API is an option because it is batch work.

Implementation specs for lower-cost models

`v2-agent-specs.md` turns Phases 0–1 into exact specs and task cards that a cheaper model can execute:

- data types, the DSL grammar and the solver algorithm, step by step;
- the diagnostic codes and their exact messages;
- worked lgh 154 fixtures with the numbers each test must reach;
- review rules, and which cards stay with a stronger model.

Phases 2–4 are outlined there, with the MCP tool schemas fixed now.

`v2-week1-pro.md` is a one-week plan for a Claude Pro subscription:

- Haiku implements and Opus (or Sonnet) reviews at four checkpoints.
- It includes the Claude Code setup files.
- Week one delivers a command-line measurement checker (solve, conflicts, missing measurements, SVG).
- At that pace, the rest takes about 10 weeks.

Effort and cost by phase (estimate, 2026-11-09)

Assumptions:

- A "session" is one focused agent task of about 1–3 hours.
- Cost per session is about $3.50 on Opus 5.5, $2.20 on Sonnet 5.5 and $0.15 on Haiku 5.5. That assumes about 4M cached input, 0.3M fresh input and 60K output per session, at the prices in "Models and cost".
- One maintainer reviews and tests. Their hours are listed separately and aren't priced.
- None of this is measured yet. Meter Phase 0 and Phase 1 and re-estimate.

Phase	Opus / Sonnet / Haiku sessions	Agent cost	Maintainer hours	Calendar
0 Spec, fixtures, reuse spikes	8 / 8 / 4	~$45	6–10	1 week
1 Kernel, solver adapter, intent DSL, CLI	30 / 25 / 5	~$160	25–40	3–4 weeks
2 Render and export (incl. `.skp`, labelled renders)	6 / 18 / 4	~$60	10–15	1.5–2 weeks
3 Checks, rooms, diagnostic views	6 / 16 / 4	~$55	10–15	1.5–2 weeks
4 Server, MCP, image import, MCP App	10 / 25 / 5	~$90	15–25	2–3 weeks
4 Eval runs (separate)	—	~$250–600	included above	in parallel
4b Accounts, sharing, MCP OAuth	1 / 8 / 1	~$20	5–8	1 week
5 Desktop editor and packaging	12 / 55 / 10	~$165	40–60	4–6 weeks
6 Mobile viewer	0 / 8 / 2	~$20	4–6	0.5–1 week
7 Import v1 and cut-over	3 / 12 / 3	~$40	8–12	1–1.5 weeks
Total	**76 / 175 / 38**	**~$650 + $250–600 evals ≈ $900–1,250**	**125–190**	**16–22 weeks**
Later: scan pipeline	6 / 15 / 3	~$55 + model tests	10–20	2–3 weeks

Range: with the usual uncertainty, about **$650–2,000** of agent and eval spend. Calendar runs 4–6 months part-time, or about 3–4 months with near-full-time review.

What changed since the first estimate ($400–1,000):

- **Down:** SolidCraft's solver replaces writing one (about −30 Opus sessions). Lifting BenchUp's snapping, input parsing, undo and app setup saves about a fifth of the editor work.
- **Up:**
  - The editor is now a full SketchUp-style tool set in Rust with packaging and signing.
  - Vision features: labelled renders, crops, diagnostic views, image import, MCP App.
  - `.skp` export through C++ and TypeScript.
  - Eval runs are now counted: about 20 tasks × 3 effort levels × 3 repeats per tier, $80–200 per full sweep, 3–5 sweeps.
- **Largest single item:** Phase 1's intent layer, the relational DSL compiled into solver constraints.
- **Biggest swing factor:** the Phase 0 solver spike. If walls don't map cleanly onto SolidCraft's sketch model, Phase 1 grows by 10–20 Opus sessions (about $35–70) and a week or two.

Running and fixed costs outside agent spend:

- **Hosting:** a small Rust server plus SQLite or libSQL, about $5–30 a month at this scale.
- **Auth provider:** a free tier at this scale.
- **Desktop code signing:**
  - Apple Developer Program, $99 a year.
  - A Windows code-signing certificate, typically about $100–400 a year.
  - Unsigned builds are possible but trigger security warnings.

- **LLM use per drawing:** about $0.02 on Haiku, $0.35 on Sonnet and $0.70 on Opus, less with caching. Each render image adds about 1–5k tokens.

Verification

- `cargo test` and `cargo clippy` in CI, for native and `wasm32` targets.
- Property tests for the kernel invariants.
- Acceptance fixtures from Phase 0 run through the CLI.
- An MCP integration test using a real client against the server, plus an agent eval comparing v1 and v2 on the same drawing task.
- Editor tests with egui's test harness. Playwright for the viewer, with a mobile viewport.
- Visual diff of rendered sheets against v1 on the fixtures.
- `.skp` exports opened in SketchUp for Web as a release check.

Risks and open questions

- **Hosting.** v1 is on Vercel. A long-lived Rust server fits Fly.io or Cloud Run. Decide before Phase 4.
- **Auth and sharing.** Decided: an auth provider in Phase 4b, not a port of v1's accounts. Pick the provider before Phase 4.
- **Solver mapping.** Walls with thickness and reference faces may not map cleanly onto SolidCraft's points and lines. The Phase 0 spike decides; the fallback is an in-house axis-aligned solver for walls, with SolidCraft kept for angled geometry.
- **Desktop distribution.** Signing and notarisation need paid developer accounts and CI secrets; unsigned builds show OS security warnings.
- **Borrowed code drift.** BenchUp and SolidCraft change daily. Vendor pinned copies with recorded source commits, keep adapters thin, and update deliberately.
- **OpenSKP writer.** Writes only the 2013–2020 container and is reverse-engineered. If SketchUp for Web rejects its files, fall back to Trimble's hosted connector.
- **egui look and accessibility.** egui doesn't look native, and its accessibility is weaker than a web UI's. Acceptable for a desktop CAD tool; the viewer stays a plain web page.
- **Rust and WASM toolchain.** Bundle size and contributor ramp-up. Keep the kernel API narrow.
- **Scan tools.** About 215 call sites in v1 tuned on golden data. They stay in v1 until the kernel is stable, so there are two codebases for a while.
- **Phases (före/efter).** They still share one geometry. A real fix (positions versioned per view) is out of scope.
- **Solver scope.** Walls stay axis-aligned first-class in the UI and the DSL. SolidCraft's solver handles general angles and arcs when they're needed.
- **Scale.** A multi-week effort with one maintainer. Each phase should be shippable on its own.